In [1]:
```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
import numpy as np
```

In [2]:
```python
data = pd.read_csv('basket.csv')
data.head(5)
```

Out[2]:

	Basket_ID	Items
0	1	bread, eggs, milk, cheese
1	2	beer, cheese, milk, diapers
2	3	milk, cheese, beer, bread, diapers
3	4	eggs, butter, bread
4	5	bread, diapers

In [3]:
```python
# Split the 'Items' column and strip any whitespace
data['Items'] = data['Items'].str.split(',').apply(lambda x: [item.strip() for item in x])
data.head(5)
```

Out[3]:

	Basket_ID	Items
0	1	[bread, eggs, milk, cheese]
1	2	[beer, cheese, milk, diapers]
2	3	[milk, cheese, beer, bread, diapers]
3	4	[eggs, butter, bread]
4	5	[bread, diapers]

In [4]:
```python
items = np.array(data['Items'])
unique_items = []

for item in items:
    for _ in item:
        unique_items.append(_)
```

In [5]:
```python
columns = np.unique(unique_items)
columns
```

Out[5]:
```
array(['beer', 'bread', 'butter', 'cheese', 'diapers', 'eggs', 'milk'],
      dtype='<U7')
```

In [6]:
```python
# Create a new DataFrame with unique items as columns
new_df = pd.DataFrame(0, index=data.index, columns=columns)

# Populate the new DataFrame with 1s where items exist in the row
for idx, items in data['Items'].iteritems():
    new_df.loc[idx, items] = 1
```

In [7]:
```python
new_df.head(5)
```

Out[7]:

	beer	bread	butter	cheese	diapers	eggs	milk
0	0	1	0	1	0	1	1
1	1	0	0	1	1	0	1
2	1	1	0	1	1	0	1
3	0	1	1	0	0	1	0
4	0	1	0	0	1	0	0

In [8]:
```python
# adding original id column
new_df['Basket_ID'] = data['Basket_ID']

#Move Basket_ID to the first column
new_df = new_df[['Basket_ID'] + [col for col in new_df.columns if col != 'Basket_ID']]
new_df.head(5)
```

Out[8]:

	Basket_ID	beer	bread	butter	cheese	diapers	eggs	milk
0	1	0	1	0	1	0	1	1
1	2	1	0	0	1	1	0	1
2	3	1	1	0	1	1	0	1
3	4	0	1	1	0	0	1	0
4	5	0	1	0	0	1	0	0

In [9]:
```python
#Frequency of each itemset
item_frequency = {}

# Calculate the frequency of each item
for column in new_df.columns[1:]:  # Skipping the first column (Basket_ID)
    total = new_df[column].sum()  # Sum the values in the column
    item_frequency[column] = total  # Add the column name and total to the dictionary
```

In [10]:
```python
item_frequency
```

Out[10]:
```
{'beer': 13,
 'bread': 20,
 'butter': 14,
 'cheese': 11,
 'diapers': 15,
 'eggs': 16,
 'milk': 17}
```

In [11]:
```python
total_transactions = len(data)  # Total number of transactions
item_support = {item: freq / total_transactions for item, freq in item_frequency.items()}

print("Support of each item:")
print(item_support)
```

```
Support of each item:
{'beer': 0.43333333333333335, 'bread': 0.6666666666666666, 'butter': 0.4666666666666667, 'cheese': 0.36666666666666664, 'diapers': 0.5, 'eggs': 0.5333333333333333, 'mil
k': 0.5666666666666667}
```

In [12]:
```python
# Generate one-item set association rules
print("\nAssociation Rules (One-item sets):")
rules = []

for i in range(len(columns)):
    for j in range(len(columns)):
        if i != j:
            item1 = columns[i]
            item2 = columns[j]

            # Support of the itemset (item1 and item2 together)
            support_both = (new_df[item1] & new_df[item2]).sum() / total_transactions

            if support_both > 0:  # Rule exists only if both items occur together
                confidence = support_both / item_support[item1]  # Confidence: P(item2 | item1)
                lift = confidence / item_support[item2]  # Lift: confidence / P(item2)

                rules.append((item1, item2, support_both, confidence, lift))

for rule in rules:
    print(f"Rule: {rule[0]} -> {rule[1]}, Support: {rule[2]:.2f}, Confidence: {rule[3]:.2f}, Lift: {rule[4]:.2f}")
```

```
Association Rules (One-item sets):
Rule: beer -> bread, Support: 0.30, Confidence: 0.69, Lift: 1.04
Rule: beer -> butter, Support: 0.13, Confidence: 0.31, Lift: 0.66
Rule: beer -> cheese, Support: 0.17, Confidence: 0.38, Lift: 1.05
Rule: beer -> diapers, Support: 0.23, Confidence: 0.54, Lift: 1.08
Rule: beer -> eggs, Support: 0.20, Confidence: 0.46, Lift: 0.87
Rule: beer -> milk, Support: 0.27, Confidence: 0.62, Lift: 1.09
Rule: bread -> beer, Support: 0.30, Confidence: 0.45, Lift: 1.04
Rule: bread -> butter, Support: 0.27, Confidence: 0.40, Lift: 0.86
Rule: bread -> cheese, Support: 0.23, Confidence: 0.35, Lift: 0.95
Rule: bread -> diapers, Support: 0.33, Confidence: 0.50, Lift: 1.00
Rule: bread -> eggs, Support: 0.40, Confidence: 0.60, Lift: 1.13
Rule: bread -> milk, Support: 0.37, Confidence: 0.55, Lift: 0.97
Rule: butter -> beer, Support: 0.13, Confidence: 0.29, Lift: 0.66
Rule: butter -> bread, Support: 0.27, Confidence: 0.57, Lift: 0.86
Rule: butter -> cheese, Support: 0.13, Confidence: 0.29, Lift: 0.78
Rule: butter -> diapers, Support: 0.23, Confidence: 0.50, Lift: 1.00
Rule: butter -> eggs, Support: 0.20, Confidence: 0.43, Lift: 0.80
Rule: butter -> milk, Support: 0.20, Confidence: 0.43, Lift: 0.76
Rule: cheese -> beer, Support: 0.17, Confidence: 0.45, Lift: 1.05
Rule: cheese -> bread, Support: 0.23, Confidence: 0.64, Lift: 0.95
Rule: cheese -> butter, Support: 0.13, Confidence: 0.36, Lift: 0.78
Rule: cheese -> diapers, Support: 0.13, Confidence: 0.36, Lift: 0.73
Rule: cheese -> eggs, Support: 0.23, Confidence: 0.64, Lift: 1.19
Rule: cheese -> milk, Support: 0.17, Confidence: 0.45, Lift: 0.80
Rule: diapers -> beer, Support: 0.23, Confidence: 0.47, Lift: 1.08
Rule: diapers -> bread, Support: 0.33, Confidence: 0.67, Lift: 1.00
Rule: diapers -> butter, Support: 0.23, Confidence: 0.47, Lift: 1.00
Rule: diapers -> cheese, Support: 0.13, Confidence: 0.27, Lift: 0.73
Rule: diapers -> eggs, Support: 0.23, Confidence: 0.47, Lift: 0.88
Rule: diapers -> milk, Support: 0.33, Confidence: 0.67, Lift: 1.18
Rule: eggs -> beer, Support: 0.20, Confidence: 0.38, Lift: 0.87
Rule: eggs -> bread, Support: 0.40, Confidence: 0.75, Lift: 1.12
Rule: eggs -> butter, Support: 0.20, Confidence: 0.38, Lift: 0.80
Rule: eggs -> cheese, Support: 0.23, Confidence: 0.44, Lift: 1.19
Rule: eggs -> diapers, Support: 0.23, Confidence: 0.44, Lift: 0.88
Rule: eggs -> milk, Support: 0.27, Confidence: 0.50, Lift: 0.88
Rule: milk -> beer, Support: 0.27, Confidence: 0.47, Lift: 1.09
Rule: milk -> bread, Support: 0.37, Confidence: 0.65, Lift: 0.97
Rule: milk -> butter, Support: 0.20, Confidence: 0.35, Lift: 0.76
Rule: milk -> cheese, Support: 0.17, Confidence: 0.29, Lift: 0.80
Rule: milk -> diapers, Support: 0.33, Confidence: 0.59, Lift: 1.18
Rule: milk -> eggs, Support: 0.27, Confidence: 0.47, Lift: 0.88
```

In [13]:
```python
# Load the dataset
market_data = pd.read_csv('Market_Basket_Optimisation.csv',header=None)

market_data.tail(5)
```

Out[13]:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
7496	butter	light mayo	fresh bread	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7497	burgers	frozen vegetables	eggs	french fries	magazines	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7498	chicken	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7499	escalope	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7500	eggs	frozen smoothie	yogurt cake	low fat yogurt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [14]:
```python
# Preprocess: Convert each row into a list of items, dropping NaN values
market_data = market_data.apply(lambda row: row.dropna().tolist(), axis=1)

# Convert the data to a DataFrame format compatible with the Eclat algorithm
market_data = pd.DataFrame({'Items': market_data})
market_data.head(5)
```

Out[14]:

	Items
0	[shrimp, almonds, avocado, vegetables mix, gre...
1	[burgers, meatballs, eggs]
2	[chutney]
3	[turkey, avocado]
4	[mineral water, milk, energy bar, whole wheat ...

In [15]:
```python
# Define a function to compute the support of an itemset
def compute_support(itemset, data):
    count = sum(1 for transaction in data['Items'] if set(itemset).issubset(set(transaction)))
    return count / len(data)
```

In [16]:
```python
# Vanilla Apriori Algorithm
def apriori(data, min_support, min_confidence):
    # Initialize variables
    frequent_itemsets = []  # List to store all frequent itemsets
    support_data = {}       # Dictionary to store support values for all itemsets

    # F0: Start with empty itemset
    Fk = [{}]  # Dummy F0 to start the loop
    k = 1      # Start with 1-itemsets

    # Generate F1: Single-item frequent itemsets
    Fk = [{item} for item in unique_items if compute_support([item], data) >= min_support]
    frequent_itemsets.extend(Fk)
    support_data.update({frozenset(itemset): compute_support(itemset, data) for itemset in Fk})

    while Fk:
        # Extend frequent itemsets (Fk-1) to generate candidate itemsets (Ck)
        Ck = []
        for i in range(len(Fk)):
            for j in range(i + 1, len(Fk)):
                # Merge itemsets if their first k-1 items are the same
                itemset1, itemset2 = list(Fk[i]), list(Fk[j])
                itemset1.sort(), itemset2.sort()
                if itemset1[:-1] == itemset2[:-1]:
                    candidate = set(itemset1) | set(itemset2)
                    # Only add if candidate is not already in Ck
                    if candidate not in Ck:
                        Ck.append(candidate)

        # Count the support of each candidate itemset
        Fk = [itemset for itemset in Ck if compute_support(itemset, data) >= min_support]

        # Update frequent itemsets and support data
        frequent_itemsets.extend(Fk)
        support_data.update({frozenset(itemset): compute_support(itemset, data) for itemset in Fk})

    # Generate association rules
    rules = []
    for itemset in frequent_itemsets:
        if len(itemset) > 1:
            for item in itemset:
                antecedent = itemset - {item}
                consequent = {item}
                support_itemset = support_data[frozenset(itemset)]
                support_antecedent = support_data[frozenset(antecedent)]
                confidence = support_itemset / support_antecedent
                if confidence >= min_confidence:
                    lift = confidence / support_data[frozenset(consequent)]
                    rules.append((antecedent, consequent, support_itemset, confidence, lift))

    return frequent_itemsets, rules
```

In [17]:
```python
# Set minimum support and confidence levels
min_support = 0.02
min_confidence = 0.3

# Run Apriori algorithm
frequent_itemsets, association_rules = apriori(market_data, min_support, min_confidence)

# Print frequent itemsets
print("Frequent Itemsets:")
for itemset in frequent_itemsets:
    print(f"  {set(itemset)}: {compute_support(itemset, market_data):.2f}")

# Print association rules
print("\nAssociation Rules:")
for rule in association_rules:
    antecedent, consequent, support, confidence, lift = rule
    print(f"  Rule: {set(antecedent)} -> {set(consequent)}, Support: {support:.2f}, Confidence: {confidence:.2f}, Lift: {lift:.2f}")
```

```
Frequent Itemsets:
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'milk'}: 0.13
  {'milk'}: 0.13
  {'eggs'}: 0.18
  {'butter'}: 0.03
  {'milk'}: 0.13
  {'eggs'}: 0.18
  {'butter'}: 0.03
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'milk'}: 0.13
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'butter'}: 0.03
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'milk'}: 0.13
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'eggs'}: 0.18
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'butter'}: 0.03
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'eggs', 'milk'}: 0.03
  {'eggs'}: 0.18
  {'milk'}: 0.13
  {'butter'}: 0.03
  {'eggs', 'milk'}: 0.03

Association Rules:
```

In [18]:
```python
# Improved Apriori Algorithm
def improved_apriori(data, min_support, min_confidence):
    """Improved Apriori algorithm to find frequent itemsets and generate association rules."""
    # Initialize variables
    frequent_itemsets = []  # List to store all frequent itemsets
    support_data = {}       # Dictionary to store support values for all itemsets

    # F0: Start with empty itemset
    Fk = [{}]  # Dummy F0 to start the loop
    k = 1      # Start with 1-itemsets

    # Generate F1: Single-item frequent itemsets
    Fk = [{item} for item in unique_items if compute_support([item], data) >= min_support]
    frequent_itemsets.extend(Fk)
    support_data.update({frozenset(itemset): compute_support(itemset, data) for itemset in Fk})

    while Fk:
        # Extend frequent itemsets (Fk-1) to generate candidate itemsets (Ck')
        Ck_prime = []
        for i in range(len(Fk)):
            for j in range(i + 1, len(Fk)):
                itemset1, itemset2 = list(Fk[i]), list(Fk[j])
                itemset1.sort(), itemset2.sort()
                # Check if k-1 prefixes are the same and h(X) < h(Y)
                if itemset1[:-1] == itemset2[:-1] and itemset1[-1] < itemset2[-1]:
                    candidate = set(itemset1) | set(itemset2)
                    Ck_prime.append(candidate)

        # Pruning: Retain only candidates with all k-1 subsets frequent
        Ck = []
        for candidate in Ck_prime:
            subsets = [candidate - {item} for item in candidate]
            if all(frozenset(subset) in support_data for subset in subsets):
                Ck.append(candidate)

        # Count support for candidates
        Fk = [itemset for itemset in Ck if compute_support(itemset, data) >= min_support]

        # Update frequent itemsets and support data
        frequent_itemsets.extend(Fk)
        support_data.update({frozenset(itemset): compute_support(itemset, data) for itemset in Fk})

    # Generate association rules
    rules = []
    for itemset in frequent_itemsets:
        if len(itemset) > 1:
            for item in itemset:
                antecedent = itemset - {item}
                consequent = {item}
                support_itemset = support_data[frozenset(itemset)]
                support_antecedent = support_data[frozenset(antecedent)]
                confidence = support_itemset / support_antecedent
                if confidence >= min_confidence:
                    lift = confidence / support_data[frozenset(consequent)]
                    rules.append((antecedent, consequent, support_itemset, confidence, lift))

    return frequent_itemsets, rules
```

In [19]:
```python
# Set minimum support and confidence levels
min_support = 0.02
min_confidence = 0.3

# Run Improved Apriori algorithm
frequent_itemsets, association_rules = improved_apriori(market_data, min_support, min_confidence)

# Print frequent itemsets
print("Frequent Itemsets:")
for itemset in frequent_itemsets:
    print(f"  {set(itemset)}: {compute_support(itemset, data):.2f}")

# Print association rules
print("\nAssociation Rules:")
for rule in association_rules:
    antecedent, consequent, support, confidence, lift = rule
    print(f"  Rule: {set(antecedent)} -> {set(consequent)}, Support: {support:.2f}, Confidence: {confidence:.2f}, Lift: {lift:.2f}")
```

```
Frequent Itemsets:
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'butter'}: 0.47
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'milk'}: 0.57
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27

Association Rules:
```

In [20]:
```python
# Recursive DFS-based function to find frequent itemsets
def extend_itemset(prefix, items, data, min_support, frequent_itemsets, support_data):
    """Recursively extend the itemset prefix to find all frequent itemsets."""
    for item in items:
        # Extend the prefix to create a new candidate
        candidate = prefix | {item}
        # Compute support for the candidate
        support = compute_support(candidate, data)

        # If the candidate is frequent, add it to the results and extend further
        if support >= min_support:
            frequent_itemsets.append(candidate)
            support_data[frozenset(candidate)] = support
            # Recursively extend the candidate
            extend_itemset(candidate, [i for i in items if i > item], data, min_support, frequent_itemsets, support_data)

# Vanilla Eclat Algorithm
def eclat(data, min_support, min_confidence):
    """Eclat algorithm to find frequent itemsets and generate association rules."""
    # Initialize variables
    frequent_itemsets = []  # List to store all frequent itemsets
    support_data = {}       # Dictionary to store support values for all itemsets

    # Start DFS with the empty prefix
    extend_itemset(set(), unique_items, data, min_support, frequent_itemsets, support_data)

    # Generate association rules
    rules = []
    for itemset in frequent_itemsets:
        if len(itemset) > 1:
            for item in itemset:
                antecedent = itemset - {item}
                consequent = {item}
                support_itemset = support_data[frozenset(itemset)]
                support_antecedent = support_data[frozenset(antecedent)]
                confidence = support_itemset / support_antecedent
                if confidence >= min_confidence:
                    lift = confidence / support_data[frozenset(consequent)]
                    rules.append((antecedent, consequent, support_itemset, confidence, lift))

    return frequent_itemsets, rules
```

In [21]:
```python
# Set minimum support and confidence levels
min_support = 0.02
min_confidence = 0.3

# Run Eclat algorithm
frequent_itemsets, association_rules = eclat(market_data, min_support, min_confidence)

# Print frequent itemsets
print("Frequent Itemsets:")
for itemset in frequent_itemsets:
    print(f"  {set(itemset)}: {compute_support(itemset, data):.2f}")

# Print association rules
print("\nAssociation Rules:")
for rule in association_rules:
    antecedent, consequent, support, confidence, lift = rule
    print(f"  Rule: {set(antecedent)} -> {set(consequent)}, Support: {support:.2f}, Confidence: {confidence:.2f}, Lift: {lift:.2f}")
```

```
Frequent Itemsets:
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'butter'}: 0.47
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'butter'}: 0.47
  {'eggs'}: 0.53
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'eggs', 'milk'}: 0.27
  {'milk'}: 0.57

Association Rules:
```

In [ ]:
```python
```

In [ ]:
```python
```

In [ ]:
```python
```

In [ ]:
```python
```